



Security Report

Abstract

Security infrastructure underpins every product decision at Orion Finance; before expanding deployment, experienced smart contract researchers validate our architecture under incentives and time horizons that complement internal review.

Alongside that program we maintain a broader security posture: least-privilege access control and key management, secrets handling and rotation, dependency and supply-chain scanning, static analysis in CI, property-based testing, aggressive fuzzing of arithmetic surfaces, meaningful code coverage, invariant monitoring, peer code review, AI-assisted review and triage (always human-verified), operational logging and alerting, incident response playbooks, and operational controls such as timelocks and emergency pause on sensitive operations.

This bounty is one layer of that framework: continuous, incentivized scrutiny from practitioners who bring different assumptions and attack models than our core team. Research prioritizes loss of user funds, insolvency, unauthorized state changes, and denial of critical user flows, with findings classified by severity (Critical through Low, plus Informational and Invalid where applicable) reflecting impact and exploitability.

This export covers 105 distinct issue(s) from the Orion Finance security research program. On the opening pages, an issue labeling overview follows the abstract, listing every GitHub label in use and how many issues carry each label (including Informational, Invalid, and any other tags). Only Critical, High, Medium, and Low findings are documented in full thereafter.

Issue labeling overview

Label	Count
high	1
medium	9
low	11
informational	12
invalid	35
duplicate	20
out-of-scope	11

Detailed submission content in this report is limited to Critical, High, Medium, and Low only.

Critical

No issue associated with this severity.

High

#24

Summary

`LiquidityOrchestrator.returnDepositFunds()` authorises callers via `config.isOrionVault(msg.sender)`, but does not check `config.isDecommissionedVault(msg.sender)`. After `completeVaultDecommissioning()` moves the vault from the active set into the decommissioned set, `isOrionVault()` returns false. Any user with an unfulfilled pending deposit can no longer cancel because the vault's internal call to `returnDepositFunds()` reverts with `NotAuthorized`.

The asymmetry is provably a bug rather than a design choice: the adjacent function `transferVaultFees()` (line 411) correctly checks both `isOrionVault` and `isDecommissionedVault`, allowing managers to claim fees on decommissioned vaults. The omission in `returnDepositFunds()` (line 402) was simply missed.

The underlying deposited by those users sits permanently in the orchestrator with no on-chain recovery path — no admin rescue function, no re-fulfillment route, and no way to re-register the vault.

Affected Contracts

- `contracts/LiquidityOrchestrator.sol` — `returnDepositFunds()` (line ~402)
- `contracts/LiquidityOrchestrator.sol` — `transferVaultFees()` (line ~411, for comparison)
- `contracts/vaults/OrionVault.sol` — `cancelDepositRequest()`
- `contracts/OrionConfig.sol` — `completeVaultDecommissioning()`

Tested commit: 68d951307bacbb85fed6c7e3ecda19b58c0f2c56

Root Cause

`returnDepositFunds()` uses a single-predicate authorisation check:

```
function returnDepositFunds(address user, uint256 amount) external {
    if (!config.isOrionVault(msg.sender)) revert ErrorsLib.NotAuthorized(); // ← BUG
    IERC20(underlyingAsset).safeTransfer(user, amount);
}
```

`isOrionVault()` queries `encryptedVaults.contains(vault)` || `transparentVaults.contains(vault)`. When `completeVaultDecommissioning()` runs, the vault is removed from both sets and added to `decommissionedVaults`. From that point forward, `isOrionVault()` returns false, and `returnDepositFunds()` reverts for any call originating from that vault.

Meanwhile, `transferVaultFees()` uses the correct two-predicate check:

```
function transferVaultFees(uint256 amount) external {
    address vault = msg.sender;
    if (!config.isOrionVault(vault) && !config.isDecommissionedVault(vault)) // ← CORRECT
        revert ErrorsLib.NotAuthorized();
    ...
}
```

The fix that was applied to `transferVaultFees` was not applied to `returnDepositFunds`.

Compounding factor: batch-size-limited fulfillment

`completeVaultDecommissioning()` is called unconditionally for decommissioning vaults during `_processSingleVaultOperations()` — it does not verify that all pending deposits were fulfilled first. With `maxFulfillBatchSize` smaller than the number of pending deposits, only a partial batch is fulfilled before the vault is decommissioned. The remaining depositors are permanently stranded.

Impact

- Severity: HIGH — direct, permanent, irrecoverable fund loss
- Trigger: Normal admin action (vault decommissioning) combined with pending deposits exceeding `maxFulfillBatchSize`
- Scope: Every user with an unfulfilled pending deposit at decommissioning time loses their entire deposit amount
- Recovery: None without a contract upgrade; no admin rescue function exists
- Scale: Loss magnitude equals the sum of all unfulfilled pending deposits, which can be arbitrarily large

Recommended Fix

Add the `isDecommissionedVault` check to `returnDepositFunds`, mirroring the existing pattern in `transferVaultFees`:

```
function returnDepositFunds(address user, uint256 amount) external {
-   if (!config.isOrionVault(msg.sender)) revert ErrorsLib.NotAuthorized();
+   if (!config.isOrionVault(msg.sender) && !config.isDecommissionedVault(msg.sender))
+       revert ErrorsLib.NotAuthorized();
    IERC20(underlyingAsset).safeTransfer(user, amount);
}
```

Additionally, `completeVaultDecommissioning()` should verify that no pending deposits remain unfulfilled before finalising the vault's decommissioned status.

Medium

#8

Summary

`LiquidityOrchestrator._processSingleVaultOperations()` calls `config.completeVaultDecommissioning(vaultAddress)` unconditionally once a decommissioning vault is processed for a single epoch batch. There is no check that:

- all pending deposit and redeem queues have been fully drained, or
- the vault has been fully liquidated into underlying.

As a result, decommissioning can finalize while the vault still has unprocessed queued requests or residual non-underlying positions.

I validated two concrete consequences:

- if pending redeem requests exceed `maxFulfillBatchSize`, later queued redeems remain stuck after the vault is removed from the active set;
- if part of the vault's marked value still sits in non-underlying portfolio assets, the vault can still be marked decommissioned and synchronous redeem can revert because `LiquidityOrchestrator` only holds the underlying cash portion.

This is not just a cosmetic lifecycle bug. The protocol can move a vault into its final “decommissioned” state while users still have unresolved exit requests or while the vault is not actually redeemable for the amount implied by its recorded `totalAssets`.

Affected Contracts

- `contracts/LiquidityOrchestrator.sol _processSingleVaultOperations()`
- `contracts/OrionConfig.sol completeVaultDecommissioning()`
- `contracts/vaults/OrionVault.sol fulfillDeposit()`
- `contracts/vaults/OrionVault.sol fulfillRedeem()`
- `contracts/vaults/OrionVault.sol redeem()`

Tested commit: `68d951307bacbb85fed6c7e3ecda19b58c0f2c56`

Prerequisites

- A vault has entered decommissioning via `removeOrionVault()` or manager removal.
- At least one of the following is true when the decommissioning vault reaches `_processSingleVaultOperations()`:
- pending requests exceed `maxFulfillBatchSize`, or
- some of the vault's value remains in non-underlying positions instead of underlying cash.

Root Cause

Decommission completion is unconditional:

```
if (config.isDecommissioningVault(vaultAddress)) {
    config.completeVaultDecommissioning(vaultAddress);
}
```

That runs immediately after one call each to `fulfillRedeem()` and `fulfillDeposit()` and after a single `updateVaultState(...)`, but there is no verification that:

- `pendingRedeem(maxFulfillBatchSize)` and `pendingDeposit(maxFulfillBatchSize)` are now globally empty, rather than just the first processable batch being consumed; or
- the updated portfolio is fully liquidated to underlying before the vault becomes synchronously redeemable.

Path 1: overflow queued redeems become orphaned

1. Set `maxFulfillBatchSize = 1`.
2. Give two users shares in the same vault.
3. Have both users request redeem.
4. Mark the vault for decommissioning.
5. Process a single decommissioning batch.
6. Observe that one redeem request is processed, the vault is marked decommissioned, and the remaining queued redeem is still present.
7. Advance time and trigger the next upkeep.
8. Observe that the remaining queued redeem is unchanged because the vault is no longer part of the active vault set.

Path 2: vault becomes decommissioned before liquidation to underlying is complete

1. Create a vault with one depositor.
2. Give the vault a portfolio position in a non-underlying ERC4626 asset and record that position in `updateVaultState(...)`.
3. Mark the vault for decommissioning.
4. Process a single decommissioning batch that leaves the non-underlying position in the portfolio.
5. Observe that the vault is marked decommissioned anyway.
6. Attempt a synchronous redeem.
7. Observe that redeem reverts because the orchestrator only holds the underlying cash portion and cannot pay the vault's recorded redeem value.

Impact

The demonstrated impact is decommissioning-state liveness failure and decommissioned-vault exit insolvency.

- Queued requests can remain stranded after the vault is removed from future epoch processing.
- A decommissioned vault can become synchronously redeemable even though part of its marked value is still held in non-underlying assets, causing redeems to fail against actual orchestrator cash.

This breaks the protocol's core decommissioning guarantee. Users can be told a vault has completed decommissioning while:

- their queued exit was never actually processed; or
- the advertised decommissioned redeem path is unusable because the vault was finalized before liquidation to underlying was complete.

Under Orion's severity framing, this is best classified as **medium**: it creates a realistic, user-facing exit failure and a meaningful settlement bug, but the validated paths do not show direct theft by an

arbitrary external attacker.

Suggested Fix

Only call `completeVaultDecommissioning()` once explicit settlement invariants hold, such as:

- no remaining pending deposit or redeem requests across the full queues, not just the first processable batch; and
- the vault portfolio is fully liquidated to underlying or otherwise made redeemable through the decommissioned vault path.

At minimum, decommission finalization should reject any state where queued requests remain or where the vault still depends on non-underlying positions for its recorded `totalAssets`.

#34

Summary

`completeAssetsRemoval()` in `OrionConfig` (line ~319) removes all decommissioning assets from the whitelist unconditionally. It is called during every `ProcessVaultOperations` minibatch (`LiquidityOrchestrator` line 482) with no check against `_failedEpochTokens`.

When a decommissioning asset's sell fails during `SellingLeg` (the token is pushed to `_failedEpochTokens`), the epoch continues. During `ProcessVaultOperations`, `completeAssetsRemoval()` permanently removes the asset from the whitelist even though it was never successfully liquidated. Meanwhile, `updateVaultState()` records the ZK proof's portfolio state, which still includes the un-sold asset shares.

Vulnerable Code

```
// LiquidityOrchestrator.sol line 480-483 – unconditional removal
} else if (currentPhase == LiquidityUpkeepPhase.ProcessVaultOperations) {
    StatesStruct memory states = _verifyPerformData(_publicValues, proofBytes, statesBytes);
    _processMinibatchVaultsOperations(states.vaults);
    config.completeAssetsRemoval(); // <-- no check for _failedEpochTokens
}
```

```
// OrionConfig.sol – unconditional whitelist removal
function completeAssetsRemoval() external onlyLiquidityOrchestrator {
    for (uint256 i = 0; i < _decommissioningAssets.length; ++i) {
        whitelistedAssets.remove(_decommissioningAssets[i]);
    }
    delete _decommissioningAssets;
    // _failedEpochTokens is in LiquidityOrchestrator – never checked here
}
```

Attack Scenario

1. Owner calls `removeWhitelistedAsset(tokenX)` → adds to `_decommissioningAssets`
2. Epoch starts. ZK prover generates sell orders to liquidate `tokenX` positions
3. During `SellingLeg`, sell of `tokenX` fails (insufficient liquidity, MEV, token pause)
4. `tokenX` pushed to `_failedEpochTokens`, commitment rebuilt, epoch continues
5. During `ProcessVaultOperations`: `updateVaultState()` records vaults still holding `tokenX` shares
6. `completeAssetsRemoval()` permanently removes `tokenX` from whitelist

7. `_failedEpochTokens` is deleted at epoch end (line 878)

Next epoch:

- `_handleStart()` → `getAllWhitelistedAssets()` omits `tokenX` → no price captured
- Vault portfolios still contain `tokenX` positions
- ZK prover cannot generate valid proofs for vaults with un-priced assets
- Affected vaults permanently excluded from epoch processing until manual re-whitelisting

Impact

Failed sell of a decommissioning asset + unconditional whitelist removal creates permanently orphaned portfolio positions. Affected vaults cannot participate in epochs — all pending deposits and redeems for those vaults are frozen.

Recommendation

Cross-reference `_failedEpochTokens` before removing decommissioning assets:

```
function completeAssetsRemoval(address[] calldata failedTokens) external onlyLiquidityOrchestrator {
    uint256 i = 0;
    while (i < _decommissioningAssets.length) {
        bool sellFailed = false;
        for (uint256 j = 0; j < failedTokens.length; ++j) {
            if (failedTokens[j] == _decommissioningAssets[i]) {
                sellFailed = true;
                break;
            }
        }
        if (sellFailed) {
            ++i; // keep in decommissioning list for next epoch
            continue;
        }
        whitelistedAssets.remove(_decommissioningAssets[i]);
        _decommissioningAssets[i] = _decommissioningAssets[_decommissioningAssets.length - 1];
        _decommissioningAssets.pop();
    }
}
```

#36

Summary

Redemptions are settled by pushing underlying from the `LiquidityOrchestrator` to the user with `safeTransfer(user, amount)`. If the underlying reverts for that recipient (USDC-style denylist), `OrionVault.fulfillRedeem` aborts the entire batch. There is no per-user try/catch. During epochs, `_processMinibatchVaultsOperations` calls `fulfillRedeem` without the same fault isolation used on sell/buy legs, so the revert can bubble out of `performUpkeep`. The orchestrator may then stay non-`idle`, `OrionConfig.isSystemIdle()` remains false, and every flow that gates on idle (deposits, redeems, cancellations, and related config actions) is blocked for all vaults on that orchestrator until the condition is cleared. The affected LP often cannot `cancelRedeemRequest` while the system is not idle, so they cannot remove themselves from the queue.

Affected contracts / code

Contract	File	Area
----------	------	------

OrionVault (via OrionTransparentVault)	contracts/vaults/OrionVault.sol	fulfillRedeem -> loop calls transferRedemptionFunds after _redeemRequests.remove
LiquidityOrchestrator	contracts/LiquidityOrchestrator.sol	performUpkeep -> _processMinibatchVaultsOperations -> _processSingleVaultOperations -> fulfillRedeem (no try/catch around vault ops)

Root cause

1. Payout assumes transfer to user always succeeds.
2. One reverting transfer fails the whole fulfillRedeem call; state rolls back for that call, but upkeep never completes, so the system can remain stuck in a non-idle phase across attempts.
3. Sell/buy minibatches use try this._executeSell / try this._executeBuy; vault redemption path does not.

Prerequisites / conditions for exploitability

- Underlying can revert when sending to a specific address due to blocklists.
- The blocklisted address has a pending redemption processed in a batch that hits fulfillRedeem.
- The address was able to deposit initially and request a redemption, but was blocked before fulfillRedeem could be called. The blacklist could be non-intentional or can also be weaponised by an attacker if they really want to, by interacting with OFAC-sanctioned entities.

Impact

- performUpkeep can fail when the epoch reaches vault operations, and fulfillRedeem hits a safeTransfer that reverts (e.g. blacklisted USDC recipient). Pending deposits and redeems stay stuck like that. Do not get fulfilled.
- The orchestrator can stay non-idle, so isSystemIdle() is false for everyone on that orchestrator.
- All LPs / vaults tied to that orchestrator: anything that requires an idle system (e.g. new requestDeposit / requestRedeem, cancels, and various config paths that check isSystemIdle) can revert until the phase is cleared.

So, the impact is critical/high. We are aware that the likelihood is medium/low. The chances of someone getting blacklisted before fulfillRedeem is executed are not high. But, as stated above, it can be done by an attacker (eg, interacting with OFAC-sanctioned entities) who made the minimum required deposit. But the impact completely bricks the system. Hence, the high severity in this case.

Suggested remediation

Wrap each transferRedemptionFunds in try/catch (or low-level call) and credit underlyingAmount to a pullable claimable[user]:

```
// Inside fulfillRedeem loop, after computing underlyingAmount:
try liquidityOrchestrator.transferRedemptionFunds(user, underlyingAmount) {
    // success
} catch {
    pendingUnderlyingClaims[user] += underlyingAmount; // + separate claim function
}
```

Summary

KBestTvlWeightedAverage does not actually rank ERC4626 assets by comparable TVL. Instead it compares each vault's raw `totalAssets()` return value directly, even though those values are denominated in each vault underlying's own units and decimals.

As a result, KBestTvlWeightedAverage can prefer a much smaller vault over a much larger one purely because the smaller vault's underlying has more decimals or because its underlying units are priced differently. The same issue distorts the stored portfolio weights because `_calculatePositions()` also uses those raw `totalAssets()` values.

Affected contracts / code

- Affected contracts:
- KBestTvlWeightedAverage
- OrionTransparentVault::submitIntent
- Protocol context showing heterogeneous ERC4626 support:
- ERC4626ExecutionAdapter supports `vaultUnderlying != protocolUnderlying`
- ERC4626PriceAdapter explicitly prices ERC4626 vaults from heterogeneous underlyings via the price registry

Root cause

The strategist pulls raw `totalAssets()` values from each whitelisted asset:

```
address[] memory assets = CONFIG.getAllWhitelistedAssets();
uint256[] memory tvls = _getAssetTVLs(assets, n);
```

And `_getAssetTVLs()` records the value returned by `IERC4626.totalAssets()` directly:

```
try IERC4626(assets[i]).totalAssets() returns (uint256 tvl) {
    tvls[i] = tvl;
}
```

The selector and the weighting logic then operate on those raw values:

```
if (!used[j] && tvls[j] > maxTVL) {
    maxTVL = tvls[j];
    maxIndex = j;
}
```

```
uint32 weight = uint32((topTvls[i] * intentScale) / totalTVL);
```

But `totalAssets()` is denominated in the vault underlying's own units. So the strategist is not comparing "TVL in a common unit"; it is comparing raw token-denominated balances from potentially different assets.

That is incompatible with the wider Orion design, which explicitly supports heterogeneous ERC4626 assets:

- ERC4626ExecutionAdapter supports `protocolUnderlying -> swap -> vaultUnderlying -> vaultShares`
- ERC4626PriceAdapter normalizes heterogeneous vault underlyings through the price registry before deriving comparable prices

So elsewhere in the protocol, heterogeneous underlyings are normalized. KBestTvlWeightedAverage skips that normalization entirely.

Prerequisites / conditions for exploitability

- A vault uses KBestTvlWeightedAverage as strategist.
- At least two whitelisted ERC4626 assets have different underlying assets.

Impact

The TVL based strategy's k selection and weights algorithm can be entirely skewed toward the wrong assets, leading to indirect fund loss for the users through suboptimal target allocation.

#52

Summary

If the Liquidity Orchestrator is paused during LiquidityUpkeepPhase.SellingLeg and later unpaused, performUpkeep may rebalance the vault using an intent submitted before the pause. This can allocate funds to assets that were appropriate at submission time but became irrelevant after the pause.

Affected contracts / code

<https://github.com/OrionFinanceAI/protocol/blob/54fc1020568bd03c13c9e2fc1b078ca4a1f300d9/contracts/LiquidityOrchestrator.sol#L412>

<https://github.com/OrionFinanceAI/protocol/blob/54fc1020568bd03c13c9e2fc1b078ca4a1f300d9/contracts/LiquidityOrchestrator.sol#L418>

Root cause

The Liquidity Orchestrator can be paused if this [\[protocol\]\(https://docs.orionfinance.ai/developer/security/protocol-invariants\)](https://docs.orionfinance.ai/developer/security/protocol-invariants) invariant is violated and detected by the off-chain system. This can happen at any point in the upkeep lifecycle because pausing is not restricted to the Idle state.

```
function pause() external onlyOwnerOrGuardian {
    _pause();
    emit EventsLib.ProtocolPaused(msg.sender);
}
```

If a vault uses KBestApyStrategy or KBestTvlWeightedAverage and has submitted an intent based on the current best APY or TVL vaults, the state is committed and the intent is recorded.

If the protocol pauses after the intent is submitted and the state commitment is created, i.e., during LiquidityUpkeepPhase.SellingLeg, rebalancing halts and the protocol remains stuck in the SellingLeg phase because performUpkeep will revert.

```
function performUpkeep(
    bytes calldata _publicValues,
    bytes calldata proofBytes,
    bytes calldata statesBytes
) external onlyAuthorizedTrigger nonReentrant ->@revert whenNotPaused {
    ...SNIP...
}
```

After the protocol is unpaused, for example after 10 days, `performUpkeep` will be called to resume execution.

It will attempt to execute the selling leg or buying leg using the previously submitted intent from the strategist, and the legs are constructed based on the portfolio configuration from before the pause (10 days earlier).

As a result, the portfolio (intent) that was correct 10 days ago may now be stale and irrelevant. Even so, rebalancing proceeds with that stale intent, allocating vault TVL to assets that may have worsened or become less performant.

Prerequisites / conditions for exploitability

- it requires, Vault have to paused at `LiquidityUpkeepPhase.SellingLeg` or `LiquidityUpkeepPhase.BuyingLeg`
- It requires, Selected asset become non-performant in paused window

Impact

The vault can rebalance into less-performing assets, which was not intended by the strategists.

#62

Summary

Authorized assessment finding.

`LiquidityOrchestrator._aggregateVaultLeaves()` commits each vault's fee model, pending deposits, pending redeems, total supply, portfolio, and intent, but it does not commit `vault.totalAssets()`.

For Orion vaults, `_totalAssets` is the NAV value used by ERC4626 conversions, fee calculations, deposit fulfillment, redeem fulfillment, and later share pricing. Omitting it means two different vault states with identical supply, queues, portfolio, and intent but different NAV produce the same epoch input commitment.

Affected Contracts

- `contracts/LiquidityOrchestrator.sol`
- `_getVaultStateData()`
- `_aggregateVaultLeaves()`
- `_buildEpochStateCommitmentAndComponents()`
- `contracts/vaults/OrionVault.sol`
- `totalAssets()`
- `fulfillDeposit()`
- `fulfillRedeem()`
- `contracts/vaults/OrionTransparentVault.sol`
- `updateVaultState()`

Tested locally with `test/BugBounty_FreshFindings_April28.test.ts`.

Root Cause

The vault leaf is built from:

```
abi.encode(
  _currentEpoch.vaultsEpoch[i],
  vaultData.feeTypes[i],
  vaultData.performanceFees[i],
  vaultData.managementFees[i],
  vaultData.highWaterMarks[i],
  vaultData.pendingRedeems[i],
  vaultData.pendingDeposits[i],
  vaultData.totalSupplies[i],
  portfolioHash,
  intentHash
)
```

`vault.totalAssets()` is not included in `VaultStateData` and is not included in the vault leaf.

This is especially important for all-underlying or sparse-portfolio vault states, where the vault's NAV exists in `_totalAssets` while the portfolio arrays can be empty.

Impact

The epoch input commitment does not uniquely bind the vault accounting state.

Consequences:

1. A proof input can be compatible with multiple different starting NAV values for the same vault.
2. Deposit, redeem, and fee calculations can be reasoned about from a NAV value that is not bound by the on-chain input commitment.
3. `ProcessVaultOperations` can later call `updateVaultState()` with a `finalTotalAssets` derived from an uncommitted starting value.
4. The protocol can accept a state transition from an ambiguous vault state, weakening the main integrity guarantee of the epoch commitment.

Recommended Fix

Include `vault.totalAssets()` in `VaultStateData` and in each vault leaf inside `_aggregateVaultLeaves()`.

The input commitment should bind every vault field that affects deposit, redeem, fee, and final NAV calculations.

#77

Summary

When a vault asset cannot be sold during `SellingLeg`, Orion records the asset in `_failedEpochTokens` and continues the epoch. If the Liquidity Orchestrator still has enough liquid USDC from the shared buffer or from other vaults' successful sells, `ProcessVaultOperations` can still fulfill that vault's redeem request at the epoch's modeled value even though the vault's own position was never liquidated.

This creates a first-mover advantage: early redeemers from an illiquid or sharply down-moving vault can exit in USDC at par, while the remaining LPs and the rest of the protocol are left holding the failed position risk.

Affected contracts / code

- `contracts/LiquidityOrchestrator.sol`

- failed sell handling: LiquidityOrchestrator.sol:720
- retry commitment reuse: LiquidityOrchestrator.sol:725
- unconditional redeem processing: LiquidityOrchestrator.sol:925
- shared USDC payout: [LiquidityOrchestrator.sol:421](#)
- contracts/vaults/OrionVault.sol
- redeem payout path: OrionVault.sol:697

Root cause

The protocol handles a failed sell as an execution failure, but it does not turn that failure into a vault-local solvency check before paying redeemers.

The key path is:

```
// LiquidityOrchestrator.sol
try this._executeSell(token, amount, sellLeg.sellingEstimatedUnderlyingAmounts[i]) {
    // successful execution, continue.
} catch {
    _failedEpochTokens.push(token);
    _recordBufferCheckpoint();
    _currentEpoch.epochStateCommitment = keccak256(
        abi.encode(_buildProtocolStateHash(), _cachedAssetsHash, _cachedVaultsHash)
    );
    emit EventsLib.EpochStateCommitted(epochCounter, _currentEpoch.epochStateCommitment);
    return;
}
```

After the failure, the epoch continues. Later, ProcessVaultOperations still processes redemptions:

```
// LiquidityOrchestrator.sol
if (pendingRedeem > 0) {
    vaultContract.fulfillRedeem(totalAssetsForRedeem);
}
```

And the actual payout comes from the shared Liquidity Orchestrator USDC balance:

```
// OrionVault.sol
liquidityOrchestrator.transferRedemptionFunds(user, underlyingAmount);

// LiquidityOrchestrator.sol
function transferRedemptionFunds(address user, uint256 amount) external {
    if (!config.isOrionVault(msg.sender)) revert ErrorsLib.NotAuthorized();
    if (amount > 0) {
        IERC20(underlyingAsset).safeTransfer(user, amount);
    }
}
```

There is no check that the redeem amount for that vault is actually backed by liquid proceeds from that vault's own sells.

Why This Matters

If the Liquidity Orchestrator does not have enough liquid USDC, the redeem transfer reverts and the protocol gets stuck. That is the already-known DoS branch.

This report is about the other branch:

- the vault's sell failed
- but the LO still has enough liquid USDC from shared sources
- so the redeem succeeds anyway

That means the failed vault's redeemer exits in cash, while the loss / illiquidity is pushed onto:

- the remaining LPs in that vault
- the shared protocol buffer
- other vaults whose successful sells supplied the liquidity

Example 1: Price Move / Slippage Scenario

Assume:

- Vault A holds 100 WETH
- At epoch snapshot, Orion values it at 10,000 USDC
- A user requests a full redeem from Vault A

Before execution, the market moves:

- 100 WETH can now only be sold for 9,000 USDC

With a normal slippage setting, the sell fails because the realized amount is too far below the epoch estimate.

Now assume the Liquidity Orchestrator still has:

- 1,000 USDC protocol buffer
- 9,500 USDC from other vaults' successful sells

Total liquid USDC in LO = 10,500

Even though Vault A's own position could only be liquidated for 9,000, `fulfillRedeem(10,000)` can still succeed because the redeem is paid from the shared LO pool.

Result:

- the Vault A redeemer gets 10,000 USDC
- the missing value is effectively absorbed by shared protocol liquidity
- the remaining LPs and other vaults are left with the downside

Example 2: ERC4626 Illiquidity Scenario

Assume:

- Vault A holds an ERC4626 lending-vault position marked at 300,000 USDC
- A user requests redeem for 80,000 USDC
- The underlying market becomes highly utilized, so only a small portion can actually be withdrawn

During `SellingLeg`, the ERC4626 unwind fails and the token is added to `_failedEpochTokens`.

If the LO still has enough liquid USDC from:

- the shared buffer, or
- successful sells in other vaults

then `fulfillRedeem(80,000)` can still succeed even though Vault A never realized that 80,000 in cash during the epoch.

This is the same issue as above, just with an external-protocol liquidity failure instead of a spot-market slippage move.

Prerequisites / conditions for exploitability

- A vault has at least one pending redemption request in the current epoch
- One or more assets backing that vault's redemption fail to sell or unwind during `SellingLeg`
- The epoch continues after the failure via `_failedEpochTokens`
- The Liquidity Orchestrator still has enough liquid USDC from the shared buffer or from other vaults' successful sells
- `ProcessVaultOperations` uses `totalAssetsForRedeem` that is not reduced to realized proceeds from the failed vault

Impact

This creates a real first-mover extraction opportunity.

An attacker or fast redeemer can monitor for:

- a vault whose sell leg is likely to fail, and
- enough shared LO liquidity elsewhere in the system

Then they redeem early and exit at the stale / modeled value while leaving the bad or illiquid position behind for everyone else.

The protocol stays live, but value is socialized across vaults.

In practical terms, this can cause:

- cross-vault liquidity leakage
- silent depletion of the shared LO buffer
- better exits for early redeemers than for later LPs
- loss transfer from the failing vault to the rest of the system

This is economically worse than a pure revert path because the protocol can continue operating while hiding the loss in shared liquidity.

Why This Is Not a Duplicate of #52 or #60

Not Issue #52

Issue #52 is pause-specific and relies on stale intent / stale prices after a pause-unpause flow.

This issue does not require pause at all. It can happen during a normal epoch from:

- a same-epoch market move that causes a sell to fail on slippage, or
- an ERC4626 / external protocol withdrawal liquidity failure

Not Issue #60

Issue #60 is the DoS branch:

- sell fails
- LO lacks enough liquid USDC

- `transferRedemptionFunds()` reverts
- protocol gets stuck

This issue is the economic socialization branch:

- sell fails
- LO has enough shared liquid USDC
- redeem succeeds anyway
- the loss is pushed onto the rest of the system instead of reverting

#82

Summary

Commit [55d5e404](#) changed `LiquidityOrchestrator._buildVaultsEpoch()` to include every transparent vault, including zero-TVL vaults.

This fixes the earlier empty-vault decommissioning problem, but it also makes empty vault spam enter epoch processing. Since `currentMinibatchIndex` is only `uint8`, enough registered vaults can make `ProcessVaultOperations` revert forever before the final vault batch is processed.

The attacker has to be a whitelisted manager, so likelihood is low. However, once triggered, the LO remains non-idle and normal user exits, cancellations, and admin recovery actions are blocked. Low likelihood combined with critical liveness impact makes this Medium severity.

Affected contracts / code

- `contracts/LiquidityOrchestrator.sol`, `currentMinibatchIndex`
- `contracts/LiquidityOrchestrator.sol`, `_buildVaultsEpoch()`
- `contracts/LiquidityOrchestrator.sol`, `_processMinibatchVaultsOperations()`
- `contracts/factories/TransparentVaultFactory.sol`, `createVault()`

Root cause

Introduced By

The issue was introduced by commit [55d5e404](#):

fix: mitigate DoS by including empty vaults in epoch accounting, delegate no-op to zkVM

The commit removed the zero-TVL filter:

```
- uint256 maxFulfillBatchSize = config.maxFulfillBatchSize();
  for (uint16 i = 0; i < allTransparent.length; ++i) {
-     address v = allTransparent[i];
-     if (IOrionVault(v).pendingDeposit(maxFulfillBatchSize) + IOrionVault(v).totalAssets() ==
    0) continue;
-     _currentEpoch.vaultsEpoch.push(v);
+     _currentEpoch.vaultsEpoch.push(allTransparent[i]);
  }
```

There is no per-manager or global limit on registered vault count.

A whitelisted manager can repeatedly call `TransparentVaultFactory.createVault()` and create many empty vaults. These vaults have no deposits, no pending requests, and no TVL.

Before commit 55d5e404, this exact LO processing DoS was still theoretically possible, but the manager had to make every vault non-empty, for example by funding each vault with at least one unit of USDC or creating a pending deposit, so the vault passed the old filter.

After commit 55d5e404, the manager no longer needs to fund the vaults at all. Empty vaults are included in vaultsEpoch directly.

The processing index is only uint8:

```
uint8 public minibatchSize;
uint8 public currentMinibatchIndex;
```

Vault operation processing increments it before checking whether the current batch is the final batch:

```
uint16 i0 = currentMinibatchIndex * minibatchSize;
uint16 i1 = i0 + minibatchSize;
++currentMinibatchIndex;

if (i1 > vaultsEpoch.length || i1 == vaultsEpoch.length) {
    i1 = uint16(vaultsEpoch.length);
    currentPhase = LiquidityUpkeepPhase.Idle;
    currentMinibatchIndex = 0;
    _nextUpdateTime = block.timestamp + epochDuration;
}
```

If vaultsEpoch.length > 255 * minibatchSize, the call for minibatch index 255 attempts to increment currentMinibatchIndex from 255 to 256, which reverts under Solidity 0.8 overflow checks before the final-batch check can reset the index.

At the default minibatchSize = 1, this requires only 256 registered transparent vaults.

If the operator configures a larger minibatchSize, the threshold becomes higher, for example 2041 vaults when minibatchSize = 8. The issue still exists because there is no vault count cap, and the index type creates a hard maximum for every possible configuration.

Why This Is Different From The Earlier Empty Vault Issue

The earlier issue was about manager removal and empty-vault decommissioning.

In the old code, a malicious manager could create many empty vaults. removeWhitelistedManager() then had to scan the full vault registry and could run out of gas. The fallback recovery path was to decommission the vaults individually, but empty vaults were skipped by _buildVaultsEpoch() because they had no pending deposits and totalAssets() == 0. That meant empty vaults could remain stuck in the active registry.

Commit 55d5e404 fixed that specific gap by making _buildVaultsEpoch() include every registered transparent vault, including zero-TVL vaults. This allows empty decommissioning vaults to reach the zkVM no-op path and later reach completeVaultDecommissioning().

This report is not saying empty vaults are skipped. It is saying the opposite.

After the fix, empty vaults are no longer skipped, so every empty vault created by a manager becomes part of vaultsEpoch and must be processed by ProcessVaultOperations. That path uses currentMinibatchIndex as a uint8, which can only represent 0 through 255.

So the failure mode changed:

Old issue:

Empty vaults are skipped, so they cannot be decommissioned through the normal epoch path.

New issue:

Empty vaults are included, but too many of them make ProcessVaultOperations exceed the uint8 minibatch index limit.

There was also a related theoretical DoS in the old code if the manager made each vault non-empty, for example by funding each one with at least one unit of USDC or creating a pending deposit. In that case, those vaults would pass the old filter and enter vaultsEpoch.

The new fix removes even that small funding requirement. A manager can now create empty vaults in a loop, leave them at zero TVL, and they still enter epoch processing.

In short:

OrionFinanceAI/protocol#58 class: empty vaults could not be cleaned up reliably.

This regression: the cleanup fix makes empty vault spam hit an unsafe uint8 processing ceiling .

Recommended Fix

The most direct fix is to limit the number of vaults a manager can create, most honest manager won't create more than 50 vault, if more vault is needed, governance can increment the limit as time goes by. A per-manager vault cap would also address the original empty-vault manager-removal griefing class.

Prerequisites / conditions for exploitability

1. The attacker is a whitelisted manager.
2. The manager creates enough empty vaults through the factory.
3. The number of transparent vaults included in the epoch is greater than $255 * minibatchSize$.
4. The epoch reaches ProcessVaultOperations.

Impact

The LO can be permanently stuck in ProcessVaultOperations.

While the system is non-idle:

1. Users cannot request deposits.
2. Users cannot request redemptions.
3. Users cannot cancel pending deposit requests.
4. Users cannot cancel pending redemption requests.
5. Owner or guardian cannot call updateMinibatchSize() because it also requires the system to be idle.

This is a protocol-wide liveness failure. The attacker is a semi-trusted whitelisted manager, which lowers likelihood, but the impact of a stuck LO is critical.

#92

Summary

In `LiquidityOrchestrator.performUpkeep()`, the `SellingLeg` branch re-initialises the global liquidity

buffer with the prover-supplied value on every entry while `currentMinibatchIndex == 0`:

```
} else if (currentPhase == LiquidityUpkeepPhase.SellingLeg) {
    StatesStruct memory states = _verifyPerformData(_publicValues, proofBytes, statesBytes);
    if (currentMinibatchIndex == 0) {
        bufferAmount = states.bufferAmount;           // re-runs on every minibatch-0 entry
        _pendingEpochProtocolFees = states.epochProtocolFees;
    }
    _processMinibatchSell(states.sellLeg);
}
```

The on-chain resume cursor `completedInCurrentMinibatch` (introduced after the reference commit, in

feat: Introduce `completedInCurrentMinibatch` ...) lets execution re-enter `currentMinibatchIndex == 0`

after a sell fails part-way through the first minibatch window. On that re-entry, `bufferAmount` is reset back to the prover value, silently discarding the realized execution-slippage deltas that the already executed sells in the same window applied via `_updateBufferAmount`. Those sells are not re-executed

(the cursor skips them), so their deltas are lost permanently and `bufferAmount` diverges from the underlying actually held by the orchestrator for the rest of the epoch and beyond.

Relationship to prior reports (why this is a new, live path — not #48/#61/#63/#69)

This is deliberately filed as a distinct issue from the closely-related cluster:

- #48 (`_pendingEpochProtocolFees` over-accrual) — closed as false positive. The triage reasoning was that the prover value is invariant across retries (derived from the frozen epoch-start commitment), so "re-reading it on every minibatch would be a no-op." That reasoning is correct for `_pendingEpochProtocolFees`, which is never mutated on-chain after the snapshot — but it is exactly why the sibling `bufferAmount = states.bufferAmount` assignment in the same `if` block IS harmful. `bufferAmount` is mutated on-chain between the two minibatch-0 reads (by `_executeSell` / `_executeBuy` → `_updateBufferAmount`). Because the prover-supplied `states.bufferAmount` is invariant and cannot encode realized slippage, re-assigning it necessarily destroys the executed deltas. In other words, the precise invariance argued in #48 is the proof of this bug for `bufferAmount`.
- #61 / #63 (partial sell/buy re-execution → phantom portfolio / double-buy) — old code. Those reported that a retry re-executed already-processed tokens. The fix for that cluster was the `completedInCurrentMinibatch` cursor, which now correctly prevents re-execution. This finding is a residual defect introduced by that fix: the cursor skips re-execution of the completed sells, but the `bufferAmount` reset still re-runs, so the completed sells' realized slippage is dropped.
- #69 (`currentMinibatchIndex` not reset → skipped tokens) — old code. Different effect (skipped remaining tokens); also addressed by the cursor rework.

To my knowledge no open or closed issue reports that, on the current code, the minibatch-0 `bufferAmount` reset discards realized slippage of already-executed sells.

Affected contracts / code

- `contracts/LiquidityOrchestrator.sol` — `performUpkeep()` `SellingLeg` branch, `bufferAmount = states.bufferAmount` guarded only by `currentMinibatchIndex == 0` (≈ L475—L481)

- contracts/LiquidityOrchestrator.sol — `_processMinibatchLeg()` resume-cursor logic via `completedInCurrentMinibatch` ($\approx$ L724—L780)
- contracts/LiquidityOrchestrator.sol — `_handleMinibatchLegFailure()` does not advance `currentMinibatchIndex` ($\approx$ L782—L789)
- contracts/LiquidityOrchestrator.sol — `_executeSell()` / `_executeBuy()` apply realized slippage via `_updateBufferAmount(actual – estimated)`
- contracts/LiquidityOrchestrator.sol — `_updateBufferAmount()` ($\approx$ L811)

Tested against: current main (default branch / in-scope per SCOPE.md). The bug requires the `completedInCurrentMinibatch` cursor, which is present on main but not on the original reference commit

68d951307bacbb85fed6c7e3ecda19b58c0f2c56.

Root cause

The buffer-baseline assignment and the slippage-delta accumulation are order-dependent operations that must

happen exactly once in sequence: set baseline once, then accumulate realized deltas. The resume cursor

breaks this invariant for the first minibatch window.

`_processMinibatchLeg` resumes a partially-completed window with `start = i0 + completedInCurrentMinibatch`:

```
uint16 batchSize = executionMinibatchSize;
uint16 i0 = currentMinibatchIndex * batchSize; // == 0 in minibatch 0
uint16 i1 = i0 + batchSize;
...
uint16 start = i0 + completedInCurrentMinibatch; // skips already-executed sells
...
for (uint16 i = start; i < i1; ++i) {
    ...
    try this._executeSell(token, amount, estimatedUnderlyingAmounts[i]) {
        ++completedInCurrentMinibatch; // success advances cursor
    } catch {
        _handleMinibatchLegFailure(token); // does NOT touch currentMinibatchIndex
        return;
    }
}
```

On failure, `_handleMinibatchLegFailure` only appends the token to `_failedEpochTokens` and rebuilds the

commitment; it leaves `currentMinibatchIndex == 0` and `completedInCurrentMinibatch > 0`.

The next

`performUpkeep` therefore re-enters the `SellingLeg` branch with `currentMinibatchIndex == 0` and re-runs

`bufferAmount = states.bufferAmount`.

Each successful `_executeSell` had already moved `bufferAmount` by its realized slippage delta (`actual – estimated`). The retry reset throws those deltas away, while the cursor guarantees the corresponding tokens are not re-processed — so the deltas are never re-applied. The prover cannot compensate: `states.bufferAmount` is derived from the frozen `initialEpochBufferAmount` anchor in pre-trade

(estimated) terms and, by the same invariance argued in #48, is identical across all retry proofs for the

epoch. Realized slippage exists only in the live `bufferAmount` storage variable, which is never folded

back into the commitment.

Prerequisites / conditions for exploitability

- `executionMinibatchSize >= 2` (set via `updateExecutionMinibatchSize`). Minibatching > 1 is the intended production configuration for amortising gas across many traded assets.
- At least one sell in minibatch-0 succeeds and a later sell in the same window reverts. Sell reverts are an explicitly supported path (the reason `_failedEpochTokens` / try-catch exist): `SlippageExceeded`, adapter revert, thin Uniswap V3 liquidity, an external ERC4626 redeem reverting on a constrained market (cf. #60/#77), etc. An adversary holding a thinly-traded whitelisted asset can deliberately trip the slippage check to force the failure.

Impact

`bufferAmount` is the protocol's single global solvency cushion: it absorbs all sell/buy slippage, gates `withdrawLiquidity` (`amount <= bufferAmount`), seeds each epoch via `initialEpochBufferAmount`, and is consumed by the prover to size redemptions/buys.

- Overstatement (lost negative deltas): the orchestrator believes it holds more underlying than it does → owner can `withdrawLiquidity` phantom buffer, and prover-sized redemptions can exceed the real balance, causing `transferRedemptionFunds` to revert (compounding the `ProcessVaultOperations` lock class in #36/#60). Path to insolvency / stuck exits.
- Understatement (lost positive deltas): real underlying becomes permanently unaccounted.
- The error is persistent and compounding — it carries into the next epoch's anchor, so repeated ordinary execution failures accumulate drift.

Suggested remediation

Set the buffer baseline only on the true first entry of the sell leg, not on every minibatch-0 step:

```
// Option A (minimal): also require the resume cursor to be at the window start.
if (currentMinibatchIndex == 0 && completedInCurrentMinibatch == 0) {
    bufferAmount = states.bufferAmount;
    _pendingEpochProtocolFees = states.epochProtocolFees;
}
```

```
// Option B: assign the baseline once at the StateCommitment -> SellingLeg transition
// and never inside the per-minibatch sell handler.
```

After a partial-then-failed minibatch-0, `completedInCurrentMinibatch > 0`, so Option A preserves the already-applied realized deltas. (`_pendingEpochProtocolFees` is invariant per #48 and harmless either way, but keeping both payload-seeded values under the same guard avoids future drift if the fee value ever becomes execution-dependent.)

Regression test: with `executionMinibatchSize >= 2`, force a mid-window sell failure after a successful sell and assert `bufferAmount == states.bufferAmount + Σ(realized deltas of completed sells)`; fuzz the failing index within minibatch-0 and across later minibatches.

Low

#11

Summary

`OrionTransparentVault.submitIntent()` can mutate vault intent during an active epoch because it has no idle-phase guard. Vault intent is part of the committed vault state used to build `_currentEpoch.epochStateCommitment`, but `LiquidityOrchestrator.performUpkeep()` keeps accepting later minibatches against the original stored commitment.

As a result, a strategist can change committed vault state during `SellingLeg`, the live epoch commitment diverges, and the orchestrator still accepts the pre-mutation commitment.

This is the same integrity class as the already validated config-mutation stale-commitment issue, but the root cause is distinct: this path is driven by vault-level strategist state mutation rather than `OrionConfig.completeAssetsRemoval()`.

Affected Contracts

- `contracts/vaults/OrionTransparentVault.sol submitIntent()`
- `contracts/LiquidityOrchestrator.sol _aggregateVaultLeaves()`
- `contracts/LiquidityOrchestrator.sol _verifyPerformData()`

Tested commit: 68d951307bacbb85fed6c7e3ecda19b58c0f2c56

Prerequisites

- The protocol has entered `SellingLeg` for an epoch.
- A transparent vault in the epoch has a strategist able to call `submitIntent()`.
- At least one alternative whitelisted asset exists so the strategist can submit a different valid intent.

Root Cause

Vault intent is part of the committed vault state:

```
(vaultData.intentTokens[i], vaultData.intentWeights[i]) = vault.getIntent();
bytes32 intentHash = keccak256(abi.encode(vaultData.intentTokens[i], vaultData.intentWeights[i]));
```

That `intentHash` is folded into `vaultLeaf`, and therefore into `_currentEpoch.epochStateCommitment`.

But `submitIntent()` is still callable mid-epoch:

```
function submitIntent(IntentPosition[] calldata intent) external onlyStrategist {
    _portfolioIntent.clear();
    ...
}
```

There is no `isSystemIdle()` check. So the strategist can change committed vault state after the epoch commitment is already fixed.

Later, `_verifyPerformData()` still accepts the original input commitment:

```
if (publicValues.inputCommitment != _currentEpoch.epochStateCommitment) {
    revert ErrorsLib.CommitmentMismatch(publicValues.inputCommitment, _currentEpoch.epochStateCommitment);
}
```



```
Commitment);  
}
```

Impact

The demonstrated impact is a vault-level proof-boundary integrity failure.

- Strategists can mutate committed vault state after the epoch commitment is fixed.
- The live commitment diverges from the stored commitment.
- Later minibatches are still accepted against the stale committed input.

This is best classified as **medium**: the validated path breaks a core correctness guarantee of the commitment boundary, but it does not independently demonstrate arbitrary theft.

Suggested Fix

Prevent strategist intent changes during active epochs, or treat them as mutations that invalidate or refresh the epoch commitment.

Any of the following would address the issue:

1. Add `isSystemIdle()` gating to `submitIntent()`.
2. Snapshot and store current intent separately for the epoch, then ignore live changes until the next epoch.
3. Refresh `_currentEpoch.epochStateCommitment` whenever committed vault state is allowed to change mid-phase.

#13

Summary

The `ChainlinkPriceAdapter` contract expects Chainlink price feeds to return valid `answeredInRound` values, however this data item is deprecated and not guaranteed to return meaningful values per Chainlink docs. This can block price provision from the Chainlink adapter, and therefore lock the `LiquidityOrchestrator` in the `Idle` state.

Vulnerability Details

Affected Contracts

- `LiquidityOrchestrator`: the `performUpkeep` function on branch `currentPhase == LiquidityUpkeepPhase.Idle`
- `PriceAdapterRegistry`: the `setPriceAdapter` and `getPrice` functions
- `ChainlinkPriceAdapter`: the `validatePriceAdapter` and `getPriceData` functions

Root Cause

The `ChainlinkPriceAdapter` uses the `answeredInRound` value returned by `latestRoundData` as part of its price freshness validation:

```
if (answeredInRound < roundId) revert ErrorsLib.StalePrice(asset);
```

However, according to the official Chainlink [documentation](#), `answeredInRound` has been deprecated:

```
answeredInRound: Deprecated - Previously used when answers could take multiple rounds to be computed
```

As a result, this field is no longer guaranteed to contain meaningful or reliable data. This limitation is also highlighted in certain feeds, such as [ARB/USD](#):

```
answeredInRound
is the round ID of the round in which the answer was computed. (Only some AggregatorV3Interface implementations return meaningful values)
```

Consequently, a feed may return an unexpected value (e.g., `answeredInRound = 0`) even when the underlying price data is correct.

With the current implementation, such cases would trigger the stale price check and cause the adapter to revert. This effectively disables all impacted Chainlink-based price feeds, despite the availability of valid price data.

As a result, this failure would prevent the `LiquidityOrchestrator` from progressing past the `Idle` phase, blocking system operations until a fix is applied and new adapters are deployed and configured.

Prerequisites

- At least one chainlink feed must return an unexpected value for the deprecated field `answeredInRound` such that `answeredInRound < roundId`
- For significant impact the `LiquidityOrchestrator` must be in phase `LiquidityUpkeepPhase.Idle`

Suggested Remediation

```
if (answer < 1) revert ErrorsLib.InvalidPrice(asset, answer);
if (updatedAt == 0) revert ErrorsLib.InvalidPrice(asset, answer);
if (startedAt > block.timestamp) revert ErrorsLib.InvalidPrice(asset, answer);
- if (answeredInRound < roundId) revert ErrorsLib.StalePrice(asset);
  if (block.timestamp - updatedAt > feedConfig.maxStaleness) revert ErrorsLib.StalePrice
(asset);
```

References

Prior art, similar vulnerabilities, relevant documentation

See the same issue reported in LoopFi's audit contest on Code4rena: <https://github.com/code-423n4/2024-07-loopfi-findings/issues/235>

#23

Summary

`UniswapV3ExecutionAdapter` sets the Uniswap V3 swap deadline to `block.timestamp` in both `buy()` and `sell()`. This does not provide meaningful expiry protection, because the deadline is derived at execution time from the current block rather than being fixed ahead of execution. As a

result, a delayed transaction can still be included at any later time.

Vulnerability Details

Affected Contracts

- UniswapV3ExecutionAdapter: buy and sell functions

Root Cause

The adapter sets:

```
deadline: block.timestamp
```

inside the same transaction that executes the swap:

In `sell()`:

```
ISwapRouter.ExactInputSingleParams memory params = ISwapRouter.ExactInputSingleParams({
    tokenIn: asset,
    tokenOut: UNDERLYING_ASSET,
    fee: assetFee[asset],
    recipient: msg.sender,
    deadline: block.timestamp,
    amountIn: amount,
    amountOutMinimum: 0,
    sqrtPriceLimitX96: 0
});
```

In `buy()`:

```
ISwapRouter.ExactOutputSingleParams memory params = ISwapRouter.ExactOutputSingleParams({
    tokenIn: UNDERLYING_ASSET,
    tokenOut: asset,
    fee: assetFee[asset],
    recipient: msg.sender,
    deadline: block.timestamp,
    amountOut: amount,
    amountInMaximum: amountIn,
    sqrtPriceLimitX96: 0
});
```

This means the deadline is not an externally chosen freshness bound. If the transaction sits in the mempool and is mined later, the contract simply uses the later block's timestamp at execution time. The deadline therefore moves with inclusion and does not actually expire the transaction based on age.

The practical impact appears in `LiquidityOrchestrator::performUpkeep()`. The function decodes `statesBytes` into `StatesStruct`, and the resulting `buyLeg` / `sellLeg` data is consumed by `_processMinibatchBuy()` and `_processMinibatchSell()` to obtain the expected swap amounts through `buyingEstimatedUnderlyingAmounts` and `sellingEstimatedUnderlyingAmounts`.

Those estimates are computed offchain and embedded into the transaction before execution. If the transaction then remains pending in the mempool for long enough, those expected amounts can become stale relative to the live market.

As a result, the protocol may execute against outdated pricing assumptions. In particular, if the live market movement is favorable for the buy/sell action resulting in positive slippage, a MEV bot can

move the pool price against the system by `slippageTolerance+positiveSlippage` to steal the profit from live market movements. `_executeBuy()` and `_executeSell()` will still accept the trade because their protection is anchored to the stale estimated amounts and the deadline protection is ineffective.

Prerequisites

The issue is always present but the impact is limited to situations when the transaction remains in the mempool enough for the market price of an asset to move in a favorable direction significantly.

Suggested Remediation

Recommended Fix

Use a proper expiration window rather than relying on `block.timestamp`. The deadline should be computed off-chain, for example as `currentTimestamp + maxAcceptableDelta`.

If `performUpkeep` fails because the deadline has passed, the off-chain system should recompute the expected underlying amounts using current prices and submit the transaction again with an updated deadline.

References

This is a very well known issue, hence the lack of a coded PoC. However if you are not convinced of the validity of the issue let me know and I will provide one. Examples:

- <https://github.com/code-423n4/2024-05-predy-findings/issues/274>
- <https://github.com/code-423n4/2023-05-venus-findings/issues/167>
- <https://github.com/sherlock-audit/2023-04-blueberry-judging/issues/145>
- <https://solodit.cyfrin.io/issues/m-02-dangerous-use-of-deadline-parameter-code4rena-particle-protocol-particle-protocol-git>
- <https://solodit.cyfrin.io/issues/deadline-set-to-blocktimestamp-zokyo-none-vaultka-markdown>
- https://solodit.cyfrin.io/issues/m-02-blocktimestamp-use-in-dex-swap-deadlines-may-cause-poor-trading-pashov-audit-group-none-hyperhyper_2025-03-30-markdown

#32

Summary

`setTargetBufferRatio` (line 287) has no `isSystemIdle()` check. `targetBufferRatio` is included in `_buildProtocolStateHash()` (line 573), which is the anchor hash inside the ZK epoch commitment.

When a sell or buy fails during `SellingLeg/BuyingLeg`, the commitment is rebuilt using `_buildProtocolStateHash()` (lines 727-729). If `targetBufferRatio` was changed between the original commitment and the failure rebuild, the new commitment uses a different ratio than what the ZK proof was originally generated against.

Vulnerable Code

```
// LiquidityOrchestrator.sol lines 287-291
function setTargetBufferRatio(uint256 _targetBufferRatio) external onlyOwner {
    if (_targetBufferRatio == 0) revert ErrorsLib.InvalidArguments();
    if (_targetBufferRatio > 500) revert ErrorsLib.InvalidArguments();
    targetBufferRatio = _targetBufferRatio;
    // NO isSystemIdle() guard!
}
```

targetBufferRatio is read live in `_buildProtocolStateHash()`:

```
// line 573
bytes32 protocolStateHash = keccak256(abi.encode(
    // ...
    targetBufferRatio, // <-- read from storage, not epoch snapshot
    // ...
));
```

Contrast with Guarded Setters

- `updateEpochDuration`: ✓ has `isSystemIdle()`
- `updateMinibatchSize`: ✓ has `isSystemIdle()`
- `updateExecutionMinibatchSize`: ✓ has `isSystemIdle()`
- `setTargetBufferRatio`: ☐ missing
- `setSlippageTolerance`: ☐ missing (see separate issue)

Impact

Owner changes `targetBufferRatio` mid-epoch → sell/buy failure occurs → commitment rebuilt with new ratio → the prover must regenerate against a commitment whose `protocolStateHash` differs from what was audited at epoch start. This breaks the invariant that epoch parameters are immutable once committed.

Recommendation

```
function setTargetBufferRatio(uint256 _targetBufferRatio) external onlyOwner {
    if (_targetBufferRatio == 0) revert ErrorsLib.InvalidArguments();
    if (_targetBufferRatio > 500) revert ErrorsLib.InvalidArguments();
+   if (!config.isSystemIdle()) revert ErrorsLib.SystemNotIdle();
    targetBufferRatio = _targetBufferRatio;
}
```

#33

Summary

`UniswapV3ExecutionAdapter.setAssetFee()` (line ~89) can be called by the guardian or owner at any time — there is no `isSystemIdle()` check. The `assetFee` mapping directly controls which Uniswap V3 pool fee tier is used for `sell()`, `buy()`, and `previewBuy()`.

During an active epoch (`SellingLeg`/`BuyingLeg`), the ZK proof was generated assuming the old fee tier's pool pricing and liquidity. Changing the fee tier mid-epoch routes swaps through a different Uniswap V3 pool with different pricing, depth, and tick spacing.

Vulnerable Code

```
// UniswapV3ExecutionAdapter.sol
function setAssetFee(address asset, uint24 fee) external onlyOwnerOrGuardian {
    if (asset == address(0)) revert ErrorsLib.ZeroAddress();
    address pool = UNISWAP_V3_FACTORY.getPool(asset, address(UNDERLYING_ASSET), fee);
    if (pool == address(0)) revert ErrorsLib.InvalidAdapter(asset);
    assetFee[asset] = fee;
    // NO isSystemIdle() check
```

```
}
```

assetFee is consumed live in every swap:

```
function sell(address asset, uint256 amount) external override returns (uint256) {
    // ...
    ISwapRouter.ExactInputSingleParams memory params = ISwapRouter.ExactInputSingleParams({
        // ...
        fee: assetFee[asset], // <-- uses LIVE value, not epoch snapshot
        // ...
    });
}
```

Impact

- Sell-leg: Routing to a 1% pool (vs intended 0.05%) with 10-100× less liquidity causes significant price impact. If slippage exceeds tolerance → forced failure → commitment rebuild → epoch stalls.
- Buy-leg: The adapter's previewBuy() queries the new pool, returns a different amountIn than the original estimate. If the new quote exceeds maxWithSlippage, the buy reverts.
- Buffer drain: Swaps within tolerance execute at worse prices, draining the protocol buffer disproportionately.

This is distinct from issues #28 (verifier/vKey swap) and #29 (strategist/intent swap) — those affect the ZK proof system, while this affects the trade execution layer.

Recommendation

```
function setAssetFee(address asset, uint24 fee) external onlyOwnerOrGuardian {
    if (asset == address(0)) revert ErrorsLib.ZeroAddress();
+   if (!CONFIG.isSystemIdle()) revert ErrorsLib.SystemNotIdle();
    address pool = UNISWAP_V3_FACTORY.getPool(asset, address(UNDERLYING_ASSET), fee);
    if (pool == address(0)) revert ErrorsLib.InvalidAdapter(asset);
    assetFee[asset] = fee;
}
```

#40

Summary

KBestTvlWeightedAverage.submitIntent() is permissionless and ranks whitelisted ERC4626 assets by a live totalAssets() snapshot. Because totalAssets() is directly increased by deposits, an attacker can flash-loan a large amount to temporarily deposit it into a whitelisted ERC4626 vault, call submitIntent(), then immediately redeem and repay the flash loan in the same transaction. As a result the selected k assets and their weights are manipulated through the temporary TVL change. Therefore vaults using KBestTvlWeightedAverage can be pushed onto stale attacker-chosen allocation targets without the attacker needing persistent capital.

Affected contracts / code

- KBestTvlWeightedAverage: _selectTopKAssets, _getAssetTVLs, submitIntent
- OrionTransparentVault::submitIntent

Root cause

The strategist is permissionless to trigger:

```
function submitIntent() external {
    address[] memory assets = CONFIG.getAllWhitelistedAssets();
    uint256[] memory tvls = _getAssetTVLs(assets, n);
    ...
    IOriionTransparentVault(vault_).submitIntent(intent);
}
```

TVL is measured from a live ERC4626 view:

```
try IERC4626(assets[i]).totalAssets() returns (uint256 tvl) {
    tvls[i] = tvl;
}
```

For ERC4626 vaults, `totalAssets()` rises immediately when new underlying is deposited. There is no smoothing, TWAP, checkpoint, or minimum holding period. So the strategist is effectively using a manipulable spot TVL oracle.

Because `submitIntent()` is permissionless, any attacker can:

1. temporarily inflate a chosen whitelisted ERC4626's `totalAssets()` using flashloaned liquidity
2. trigger `submitIntent()`
3. let the vault store the manipulated token set and weights
4. remove the temporary TVL immediately afterward to repay the flashloan

The attacker can perform the manipulation on multiple assets such that the full result of the targets and weights is manipulated.

Prerequisites / conditions for exploitability

- A vault uses `KBestTvlWeightedAverage` as strategist.
- The injected TVL is large enough to move the target asset into the strategist's top-k set or materially increase its weight.

Impact

Indirect loss of funds for users through suboptimal target allocation due to the strategy's results being fully controlled by an attacker via flashloans.

#43

Summary

Buy execution is bounded using an on-chain quote that is fetched at execution time from the live Uniswap pool state. If an attacker can reliably frontrun/backrun upkeep buys, they can first move the pool into a manipulated state so the adapter's `previewBuy()` returns an inflated input amount, then let the protocol accept the buy near the top of its allowed spend band (allowed approved amount). Because `LiquidityOrchestrator` compares the actual spend against the off-chain estimate, each manipulated buy reduces `bufferAmount`. Repeating this process can drain the buffer over time. Once the remaining buffer is too small to absorb another adverse buy delta, the buy reverts inside `_updateBufferAmount`, the token is added to `_failedEpochTokens`, and the protocol needs additional upkeep/proof cycles to continue, wasting gas and degrading execution.

Affected contracts / code

Contract	File	Area
LiquidityOrchestrator	contracts/LiquidityOrchestrator.sol	<code>_executeBuy</code> -> compares actual spend against off-chain estimate and socializes the delta to buffer
LiquidityOrchestrator	contracts/LiquidityOrchestrator.sol	<code>_updateBufferAmount</code> -> negative buy deltas reduce buffer and can underflow
LiquidityOrchestrator	contracts/LiquidityOrchestrator.sol	<code>_processMinibatchBuy</code> > failed buys are added to <code>_failedEpochTokens</code> , forcing more upkeep cycles
UniswapV3ExecutionAdapter	contracts/execution/UniswapV3ExecutionAdapter.sol	<code>buy</code> -> recomputes input amount from live <code>previewBuy()</code> quote at execution time

Root cause

1. Live on-chain quote used during execution: the adapter uses `previewBuy()` against the current pool state when the buy is already being executed.
2. Execution compared to an off-chain estimate: the protocol accepts buys using a spend cap informed by a manipulated live quote, but charges the deviation against the original off-chain estimate.
3. Buffer absorbs adverse execution: when `executionUnderlyingAmount` > `estimatedUnderlyingAmount`, the difference is subtracted from `bufferAmount`.
4. Failure path increases upkeep churn: once the buffer cannot absorb the next manipulated delta, the token is marked failed and the protocol must spend additional upkeep/proof cycles to continue.

Prerequisites / conditions for exploitability

- The attacker can reliably sandwich upkeep-triggered buy transactions for a target pool.
- The manipulated price remains inside the protocol's configured acceptance band/approval band for that buy.
- The protocol has non-zero `bufferAmount` to absorb adverse execution initially.

Impact

Medium: a searcher who can repeatedly sandwich upkeep buys can systematically force accepted buy executions above the protocol's off-chain estimate, causing `bufferAmount` to decrease on each manipulated buy. This means a buy is always bound to have a negative delta for the buffer update. The delta will never be positive because of the attack. The buffer is almost like an incentive for the attacker. Over time, this can exhaust the buffer, after which further manipulated buys fail and get marked as failed tokens. The epoch can still progress eventually, but only after additional upkeep/proof cycles, wasting gas and causing partial or skipped purchases. Because `depositLiquidity()` is only callable when the system is idle, the protocol cannot replenish the buffer mid-epoch.

Yes, the attack is capped by the approved amount. The attacker can not manipulate beyond the approved or slippage amount, which does not make this a high-severity issue. But the core issue is that the protocol always executes the buy in a manipulated pool. Using `preview-buy` basically gets the spot price, which is prone to manipulation. The delta added to the buffer will hence always be incentivised to be negative.

Remediation

Do not use on-chain `previewBuy` to determine `amountIn`, as it's essentially the spot price. Rely on the TWAP price or the oracle price of the asset. Generate the amount to be traded using this TWAP price. Sandwich attacks cannot manipulate the TWAP price.

References

1. <https://dacion.me/defi-slippage-attacks#heading-on-chain-slippage-calculation-can-be-manipulated>
2. <https://github.com/sherlock-audit/2023-01-derby-judging/issues/310>

#46

Summary

`KBestApyStrategist` stores share-price checkpoints in `_checkpoints[asset]`. Nothing clears an entry from checkpoints when an asset leaves `CONFIG.getAllWhitelistedAssets()`. After `remove` → long gap → re-add, the next `submitIntent` can still read a checkpoint whose `timestamp` and `sharePrice` were recorded before the asset was delisted. `_getAssetApy` then uses a large `elapsed` that includes the entire period the token was not in the protocol investment universe, which won't be the case for other tokens as their `elapsed` values would be less (assuming that `submitIntent` has been called in between delisting and relisting), so the annualized figure used for top-K selection is economically stale / misleading relative to "APY since we re-listed this token."

```
function _getAssetApy(address asset) internal view returns (uint256) {
    Checkpoint memory cp = _checkpoints[asset];
    // slither-disable-next-line incorrect-equality
    if (cp.sharePrice == 0 || cp.timestamp == 0) return 0;

    uint256 elapsed = block.timestamp - cp.timestamp;
    if (elapsed < MIN_WINDOW) return 0;

    uint256 currentPrice = _getSharePrice(asset);
    if (currentPrice == 0 || currentPrice > type(uint128).max) return 0;
    if (currentPrice < cp.sharePrice) return 0;

    uint256 delta = currentPrice - uint256(cp.sharePrice); //@audit - big delta value
    uint256 tmp = Math.mulDiv(delta, WAD, uint256(cp.sharePrice));
    return Math.mulDiv(tmp, SECONDS_PER_YEAR, elapsed);
}
```

Affected contracts / code

`contracts/strategies/KBestApyStrategist.sol` -> `_getAssetApy`

Root cause

1. Persistent map: `_checkpoints` is write-only for active assets during `submitIntent`; delisting does not delete keys.
2. Mismatched time base: `_getAssetApy` annualizes from the last stored checkpoint to now using `elapsed = now - cp.timestamp`, even if the asset spent part of that window off `getAllWhitelistedAssets()` (no checkpoint refresh, but the ERC4626 price can still move). After re-list, that blends off-list price path with a long denominator, so ranked APY is not "like-for-like since re-list" vs peers whose checkpoints stayed current while listed.

Prerequisites / conditions for exploitability

- An asset was previously whitelisted and had at least one checkpoint recorded while linked to this strategist's vault.
- The asset is removed from the config whitelist (full removal after decommissioning flow) and later whitelisted again.
- submitIntent runs after re-list with ranking driven by _getAssetApy / top-K selection.

Impact

Medium (strategy / rebalancing economics): The issue is that submitIntent can produce a mis-stated allocation because APY is computed from a stale checkpoint and elongated time window. That can materially distort top-K ranking, the resulting weight share for a re-listed asset, steering downstream rebalancing toward an intent vector that does not reflect a fair post—re-list comparison.

Remediation

After re-list, force a fresh baseline for that asset (overwrite checkpoint).

#56

Summary

OrionVault models pending deposits and redeems as queues, but implements them with EnumerableMap.AddressToUintMap, which explicitly makes no ordering guarantees. When a request is fulfilled, EnumerableMap.remove() delegates to EnumerableSet._remove(), which uses swap-and-pop and therefore mutates the order of the remaining entries.

Orion tries to avoid this by snapshotting the current batch into memory before iterating, but that only protects the users already selected for the current batch. It does not preserve the order of the remaining queue after those removals happen.

As a result, once maxFulfillBatchSize < queue length, a later request can jump ahead of earlier requests across epoch boundaries. In the redeem path, that means a user who requested last can exit before earlier redeemers and avoid a later loss that should have hit them.

Affected Contracts

- contracts/vaults/OrionVault.sol
- node_modules/@openzeppelin/contracts/utils/structs/EnumerableMap.sol
- node_modules/@openzeppelin/contracts/utils/structs/EnumerableSet.sol

Tested commit: 68d951307bacbb85fed6c7e3ecda19b58c0f2c56

Root Cause

The root cause is using an unordered swap-and-pop map as if it were a FIFO queue.

Relevant code:

1. Pending requests are read by positional index using .at(i). - contracts/vaults/OrionVault.sol:594-643
2. Fulfillment snapshots only the current batch, not the remainder of the queue. - contracts/vaults/OrionVault.sol:657-694 - contracts/vaults/OrionVault.sol:697-722

3. OpenZeppelin documents that EnumerableMap makes no ordering guarantees. - node_modules/@openzeppelin/contracts/utils/structs/EnumerableMap.sol:14-19
4. OpenZeppelin removal uses swap-and-pop and explicitly modifies array order. - node_modules/@openzeppelin/contracts/utils/structs/EnumerableSet.sol:99-116

The comment in `fulfillRedeem()` says keys are collected first "to avoid swap-and-pop reordering issues", but that only avoids reordering inside the same batch. It does not make the overall queue FIFO across multiple fulfill calls.

Preconditions

1. `maxFulfillBatchSize` is smaller than the number of pending redeem requests.
2. The protocol fulfills requests across multiple epochs or minibatches.
3. A material NAV change happens between those batches.

Those conditions are normal protocol behavior, not an artificial edge case.

Result

The PoC passes and shows two things:

1. After processing the first redeem request from [U1, U2, U3, U4] with batch size 1, the remaining order becomes [U4, U2, U3].
2. A loss applied after U4 exits causes U4 to receive more underlying than the earlier redeemers U2 and U3.

Impact

This is a real priority-theft / value-shift bug.

1. Later redeemers can jump ahead of earlier redeemers and exit before adverse NAV changes.
2. The same root cause also applies to deposit queues, where later depositors can jump ahead of earlier depositors before favorable NAV changes.
3. Because fulfillments are intentionally batch-limited, this ordering corruption is not theoretical; it appears exactly when the queue spans multiple batches.

The protocol therefore does not provide FIFO fulfillment semantics even though the code structure, comments, and request model imply queue behavior.

Why This Is Distinct From Existing Issues

I did not find a live bounty issue through #55 covering request-order corruption, queue leapfrogging, or `EnumerableMap` swap-and-pop priority theft in deposit or redeem fulfillment.

This is also distinct from the in-repo batch-limit accounting tests. Those tests and fixes focus on amount consistency and overcounting when only part of the queue is fulfilled. This report is about fairness and economic ordering: the remaining queue itself is mutated, allowing later users to jump ahead of earlier ones.

Suggested Fix

1. Replace `EnumerableMap`-based request storage with a true FIFO queue structure.
2. If $O(1)$ deletions are needed, maintain an explicit linked-list / ring-buffer / indexed queue rather than relying on unordered enumerable maps.
3. If keeping the current storage shape, preserve a stable ordering key and always fulfill in ascending insertion order rather than by current array position.

Severity Rationale

Medium is appropriate because the bug can misallocate redemption outcomes between honest users during normal multi-batch processing, especially around sharp NAV moves.

#58

Summary

OrionConfig:removeWhitelistedManager also tries to decommission all vaults related to the manager. If the manager has created many empty vaults, the function must loop through them all, which can run out of gas. Please note that this is different from #45](#45) .

Affected contracts / code

<https://github.com/OrionFinanceAI/protocol/blob/c0811c2c68de1c889541d795f51a54eb39b2c518/cntracts/OrionConfig.sol#L366-L367>

<https://github.com/OrionFinanceAI/protocol/blob/c0811c2c68de1c889541d795f51a54eb39b2c518/cntracts/LiquidityOrchestrator.sol#L563>

Root cause

When the protocol wants to remove a manager, it calls removeWhitelistedManager. In addition to removing the manager, it also tries to decommission every vault associated with them. To find those vaults, it loops through all transparentVaults in the Orion system, which can be large.

```
/// @inheritdoc IO OrionConfig
function removeWhitelistedManager(address manager) external onlyOwner {
    if (!this.isWhitelistedManager(manager)) revert ErrorsLib.InvalidAddress();
    if (!isSystemIdle()) revert ErrorsLib.SystemNotIdle();

    // Decommission all vaults associated with this manager
    // Check transparent vaults
    address[] memory transparentVaultsList = this.getAllOrionVaults(EventsLib.VaultType.Transparent);
    for (uint256 i = 0; i < transparentVaultsList.length; ++i) {
        address vault = transparentVaultsList[i];
        if (IO OrionVault(vault).manager() == manager) {
            // Mark vault for decommissioning
            // slither-disable-next-line unused-return
            decommissioningInProgressVaults.add(vault);
            IO OrionVault(vault).overrideIntentForDecommissioning();
            emit EventsLib.VaultDecommissioningInitiated(vault);
        }
    }
}
```

Once the manager is whitelisted, they can create many empty vaults (totalAssets=0) using TransparentVaultFactory:createVault, which increases the size of transparentVaults[]. If enough vaults are created, the loop inside removeWhitelistedManager can be DoSed due to an out-of-gas issue. Fusaka also introduced a 16 million gas per-transaction limit, which is much lower than the previous 30 million block gas limit threshold.

It can be argued that this can be mitigated by calling removeOrionVault individually. However, if the vault is empty, it is never decommissioned because, when building _currentEpoch, the protocol ignores vaults with 0 assets.

The empty vault never propagates further and completeVaultDecommissioning is never called for empty vaults.

```

/// @notice Build filtered transparent vaults list for the epoch
function _buildVaultsEpoch() internal {
    address[] memory allTransparent = config.getAllOrionVaults(EventsLib.VaultType.Transparent);
    delete _currentEpoch.vaultsEpoch;

    uint256 maxFulfillBatchSize = config.maxFulfillBatchSize();
    for (uint16 i = 0; i < allTransparent.length; ++i) {
        address v = allTransparent[i];
        @audit -> Ignore empty vault
        if (IOrionVault(v).pendingDeposit(maxFulfillBatchSize) + IOrionVault(v).totalAssets
        () == 0) continue;
        _currentEpoch.vaultsEpoch.push(v);
    }
}

```

The known issue [KI-003](#) discusses how `performUpkeep` can run out of gas, but it does not mention this specific scenario of how removal of a malicious manager can also be DoSed. The fix [here](#) also does not handle this case. So I believe this is a unique issue.

Prerequisites / conditions for exploitability

- manager becomes malicious and they should have large number of empty vault

Impact

A manager who cannot be removed retains indefinite control over their vaults' portfolio strategy. They can submit intents that direct user deposits toward illiquid or adversarial assets, causing losses to depositors. The attack requires no ongoing cost beyond the initial vault creation transactions.

#59

Summary

[ERC-4626](#) requires that `maxDeposit`, `maxMint`, `maxRedeem`, and `maxWithdraw` factor in all global and user-specific limits and return 0 whenever the corresponding action is entirely disabled, even temporarily. `OrionVault` inherits these four functions unchanged from `OpenZeppelin's ERC4626Upgradeable` and never overrides them. As a result, all four return values that are inconsistent with what the actual entry points (`requestDeposit`, `requestRedeem`, `redeem`) enforce, breaking the contract's ERC-4626 interface guarantee.

Affected contracts / code

<https://github.com/OrionFinanceAI/protocol/blob/main/contracts/vaults/OrionVault.sol>

Root cause

`OrionVault` does not override `maxDeposit`, `maxMint`, `maxRedeem`, or `maxWithdraw`. The inherited defaults are:

Function	Default return value
<code>maxDeposit</code>	<code>type(uint256).max</code>
<code>maxMint</code>	<code>type(uint256).max</code>
<code>maxRedeem</code>	<code>balanceOf(owner)</code>
<code>maxWithdraw</code>	<code>convertToAssets(balanceOf(owner))</code>

Each of these contradicts constraints enforced elsewhere in the contract:

maxDeposit / maxMint return type(uint256).max but requestDeposit reverts when:

- !config.isSystemIdle() — deposits blocked for the entire duration of every active epoch
- isDecommissioning — deposits permanently blocked for this vault
- depositAccessControl rejects the caller — user-specific block
- assets < config.minDepositAmount() — protocol floor not visible to callers

maxRedeem returns balanceOf(owner) but requestRedeem reverts when:

- !config.isSystemIdle() — redemption requests blocked mid-epoch
- shares < config.minRedeemAmount() — a user whose balance is below the minimum sees maxRedeem = balance but every call reverts

maxWithdraw returns a non-zero value but withdraw() unconditionally reverts:

```
// OrionVault.sol L225
function withdraw(uint256, address, address) public pure override returns (uint256) {
    revert SynchronousCallDisabled();
}
```

Mitigation

Override all four functions to reflect actual protocol state:

```
function maxDeposit(address receiver) public view override returns (uint256) {
    if (!config.isSystemIdle()) return 0;
    if (isDecommissioning || config.isDecommissionedVault(address(this))) return 0;
    if (depositAccessControl != address(0) &&
        !IOrionAccessControl(depositAccessControl).canRequestDeposit(receiver)) return 0;
    return type(uint256).max;
}

function maxMint(address receiver) public view override returns (uint256) {
    uint256 maxAssets = maxDeposit(receiver);
    if (maxAssets == 0) return 0;
    return convertToShares(maxAssets);
}

function maxRedeem(address owner) public view override returns (uint256) {
    if (!config.isSystemIdle()) return 0;
    if (config.isDecommissionedVault(address(this))) return 0;
    uint256 shares = balanceOf(owner);
    if (shares < config.minRedeemAmount()) return 0;
    return shares;
}

function maxWithdraw(address owner) public view override returns (uint256) {
    uint256 maxShares = maxRedeem(owner);
    if (maxShares == 0) return 0;
    return convertToAssets(maxShares);
}
```

Prerequisites / conditions for exploitability

- A third-party protocol or aggregator integrates OrionVault as a standard ERC-4626 vault and reads any of the four max* functions before transacting
- For maxDeposit / maxRedeem: the protocol is in an active epoch (SellingLeg, BuyingLeg, ProcessVaultOperations) — the normal state for a large portion of every cycle

- For `maxWithdraw`: no prerequisites — `withdraw()` unconditionally reverts in all states, so `maxWithdraw` is always wrong
- For `maxDeposit` (access control case): vault manager has set a non-zero `depositAccessControl` contract
- For `maxRedeem` (minimum case): user share balance is below `config.minRedeemAmount()`

Impact

Any protocol, aggregator, or off-chain integration that follows the ERC-4626 interface will read these functions to determine safe call amounts before transacting. Because the return values are wrong, integrators will:

- Attempt deposits during active epochs and receive unexpected reverts
- Attempt redeems with balances below `minRedeemAmount` and receive unexpected reverts
- Attempt any call to `withdraw()` after reading a non-zero `maxWithdraw`

The vault is explicitly built to be composable (ERC-4626 compliance is listed as a design goal in the NatSpec). The mismatch between advertised limits and enforced limits silently breaks every downstream integration that relies on the standard interface.